Chapter 09: Spring email support

Department of Software Engineering
Faculty of Information Technology (HCM-UTE)
Course: Web Programming
Course ID: WEPR330479
Lecturer: Msc. Khoa Hoang

Content

- Email fundamentals
- SMTP Configuration via the application.properties file
- Sending plain & rich emails
- Templating with Thymeleaf
- Asynchronous email delivery

Email fundamentals

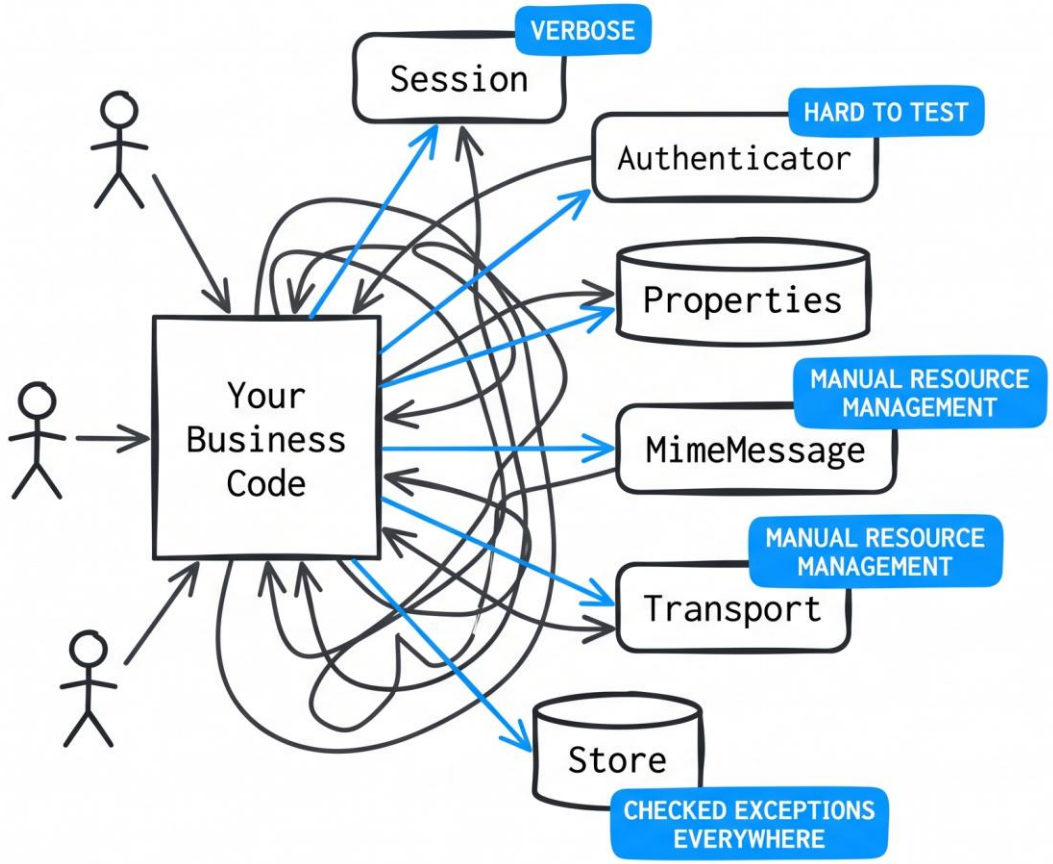
Connecting web applications to SMTP mail services

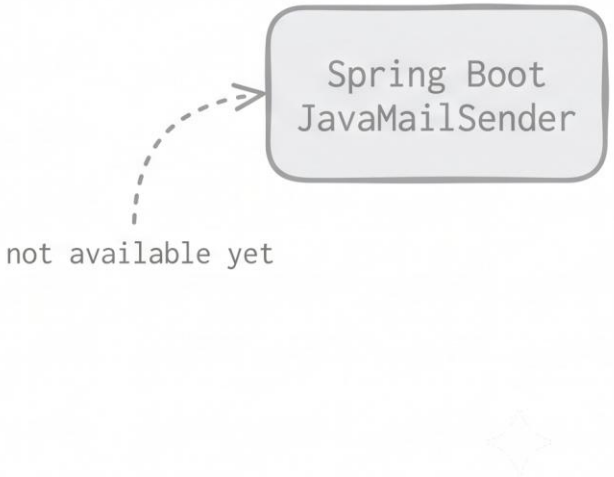
Why need to send emails in web applications?

- Account activation and email verification links
- Secure password reset workflows
- Transactional notifications like order receipts and invoices
- System alerts and administrator reports
- …

Standard JavaMail

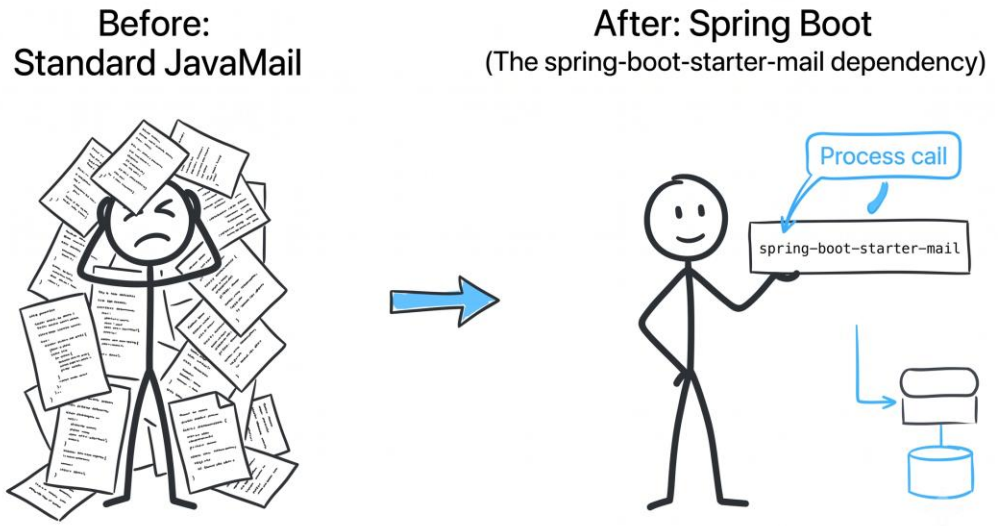
- Standard JavaMail requires verbose Session and Transport code





Spring Mail

- Spring Mail provides high-level via JavaMailSender interface
- Eliminates boilerplate connection and cleanup code
- Official documentation: https://docs.spring.io/spring-framework/reference/integration/email.html



Spring Boot Mail starter

- The *spring-boot-starter-mail* dependency adds Jakarta Mail and Spring support
- Auto-configures a JavaMailSender bean from application properties
- Inject JavaMailSender directly into your service classes
- Zero XML configuration required

SMTP Configuration

Connecting to mail servers via application.properties

Core SMTP properties

- spring.mail.host: SMTP server address (e.g., smtp.gmail.com)
- spring.mail.port: standard ports like 587 (STARTTLS) or 465 (SSL)
- spring.mail.username: sender email address or API username
- spring.mail.password: email password or dedicated app password

Mail servers and sandbox testing

- Production: Gmail, SendGrid, Amazon SES or custom SMTP
- Development sandbox: Mailpit, Mailtrap or MailHog intercepts emails safely
- Prevents accidental spamming of real user inboxes during labs
- Inspect email headers, HTML rendering and spam score in browser

Security: TLS and credentials

- spring.mail.properties.mail.smtp.auth=true: require authentication
- spring.mail.properties.mail.smtp.starttls.enable=true: encrypt channel
- Never commit plain credentials to version control
- Load passwords via environment variables or Spring profiles

Sending plain & rich emails

From simple text messages to multipart MIME emails

SimpleMailMessage #1

- SimpleMailMessage is the simplest class in the Spring Framework for sending plain-text emails (text only, no HTML, no attachments).
- It is a convenient wrapper around the JavaMail API that allows you to send emails with just a few lines of code, rather than having to set up a MimeMessage, Session, and Transport as required by standard JavaMail.
- Set recipients: setTo(), setCc(), setBcc()
- Set subject and content: setSubject(), setText()
- Fastest to send with minimal overhead

SimpleMailMessage #2

```
@Service
public class EmailService {
    private final JavaMailSender mailSender;

    public EmailService(JavaMailSender mailSender) {
        this.mailSender = mailSender;
    }

    public void sendSimpleEmail(String to, String subject, String body) {
        SimpleMailMessage message = new SimpleMailMessage();
        message.setFrom("noreply@example.com");
        message.setTo(to);
        message.setSubject(subject);
        message.setText(body);

        mailSender.send(message);
    }
}
```

Sending a simple email in just five lines, that is the value of SimpleMailMessage.

MimeMessage

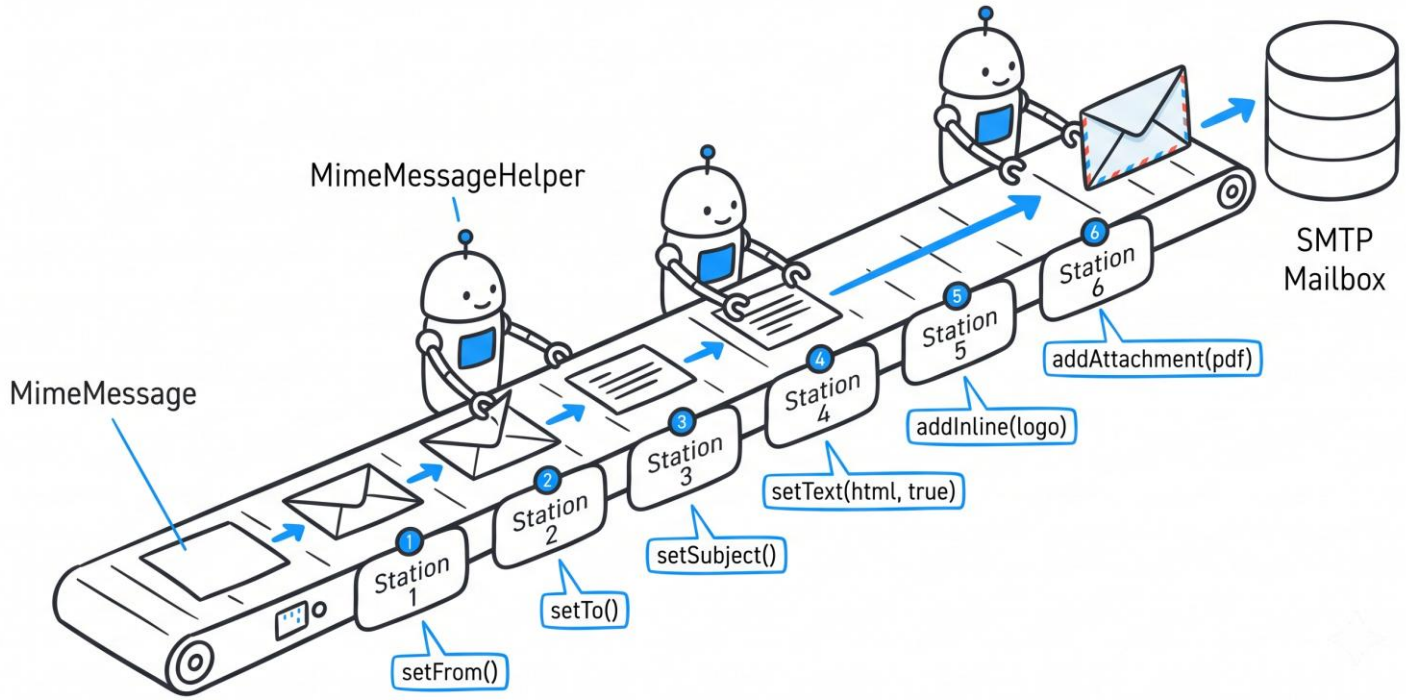
- MimeMessage supports rich HTML formatting and attachments

- MIME (Multipurpose Internet Mail Extensions) is an email extension standard that allows
  - Send HTML instead of just plain text
  - Attach files (PDF, Excel, images, etc.)
  - Embed images inline within the HTML content
  - Support multiple encodings (UTF-8, ISO-8859-1, etc.)
  - Send multipart messages within a single email

- SimpleMailMessage cannot do these things, so we need to use MimeMessage.

MimeMessage and MimeMessageHelper

- MimeMessage is the actual email object that gets sent, whereas MimeMessageHelper is a tool that allows you to populate that object with content without having to deal with the complex JavaMail API.



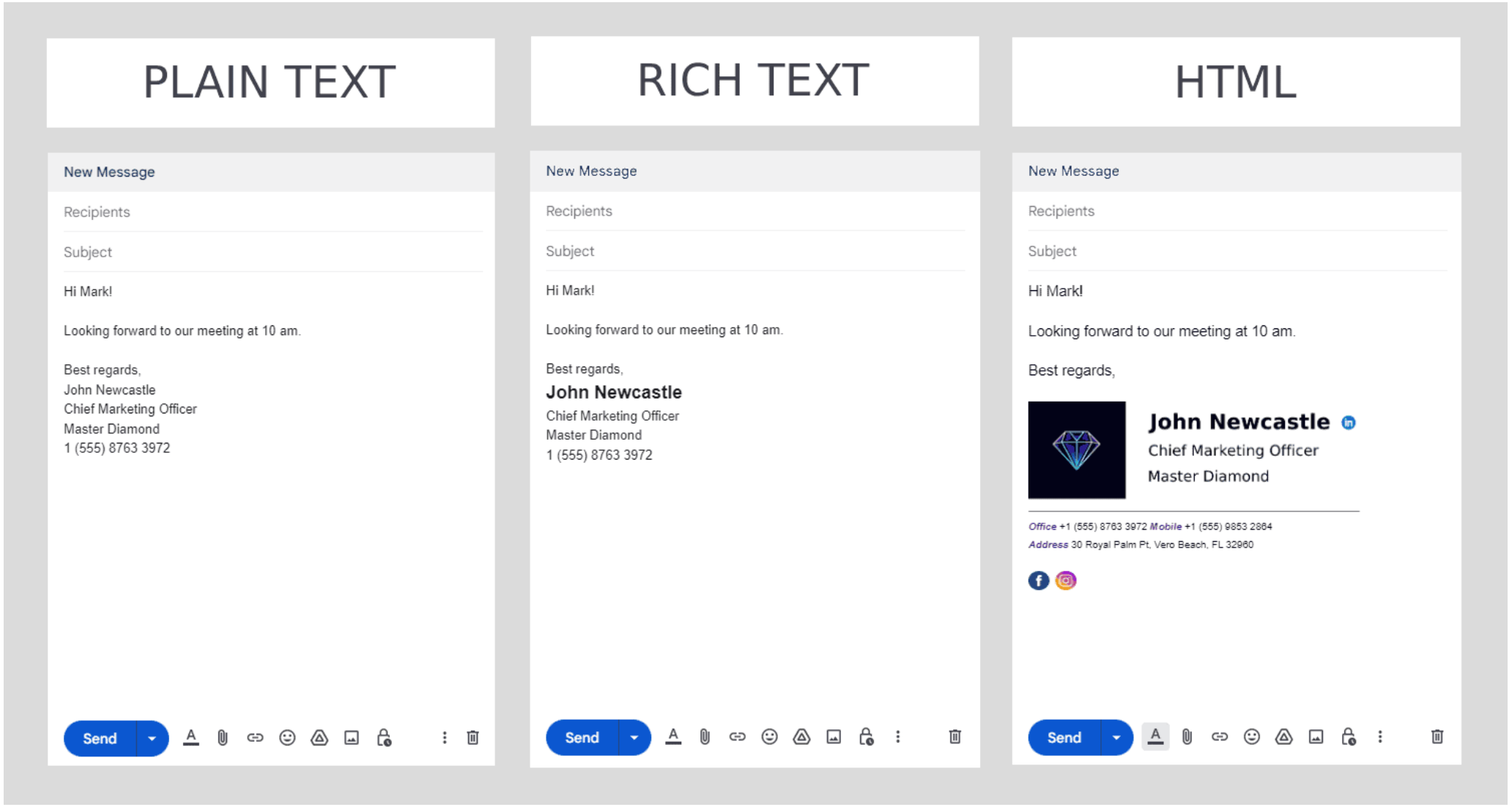
Templating with Thymeleaf

Generating dynamic and branded HTML emails

Why email templating?

- Hardcoding HTML strings in Java code is messy and unmaintainable
- Templates allow clean HTML/CSS separation from Java logic
- Supports dynamic variables, loops and conditions
- Reuses Thymeleaf engine already in project.

Example


PLAIN TEXT
New Message
Recipients
Subject
Hi Mark!
Looking forward to our meeting at 10 am.
Best regards,
John Newcastle
Chief Marketing Officer
Master Diamond
1 (555) 8763 3972
Send
RICH TEXT
New Message
Recipients
Subject
Hi Mark!
Looking forward to our meeting at 10 am.
Best regards,
John Newcastle
Chief Marketing Officer
Master Diamond
1 (555) 8763 3972
Send
HTML
New Message
Recipients
Subject
Hi Mark!
Looking forward to our meeting at 10 am.
Best regards,
John Newcastle
Chief Marketing Officer
Master Diamond
Office +1 (555) 8763 3972 Mobile +1 (555) 9853 2864
Address 30 Royal Palm Pt, Vero Beach, FL 32960
Send

SpringTemplateEngine integration

- Spring Boot auto-configures SpringTemplateEngine
- Create org.thymeleaf.context.Context and set variables
- templateEngine.process('mail/template', context) returns HTML string
- Pass processed string directly to MimeMessageHelper

Designing email templates

- Place email templates under src/main/resources/templates/mail/
- Use inline CSS styles; many email clients strip <style> tags
- Keep layouts simple and responsive
- Test across desktop clients, webmail and mobile apps

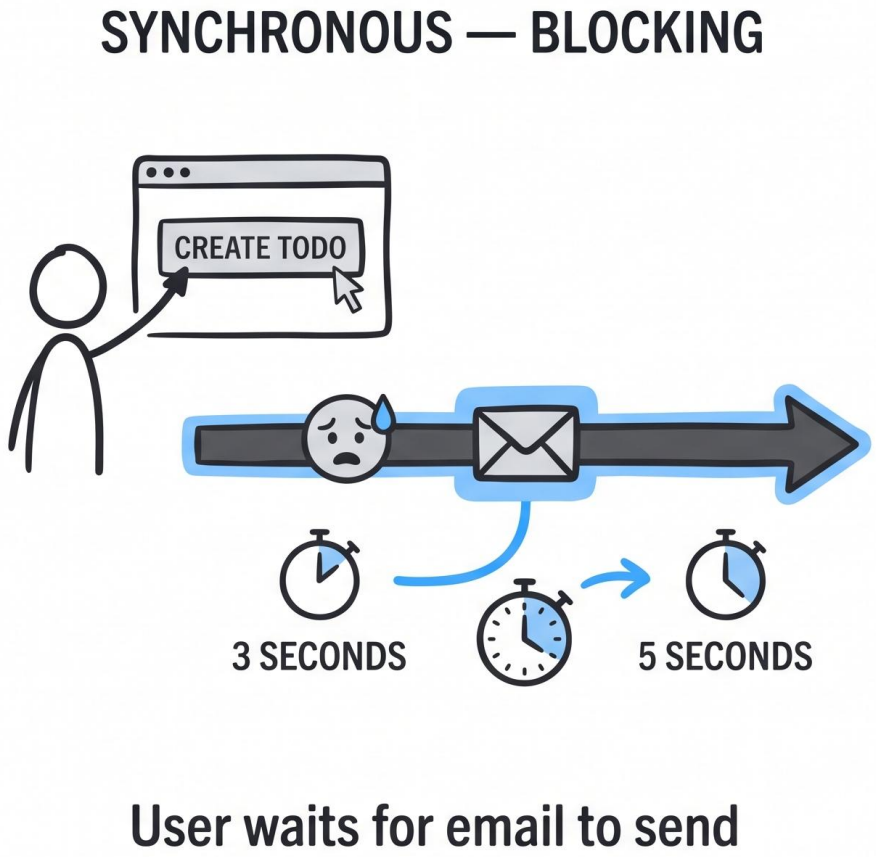
Asynchronous email delivery

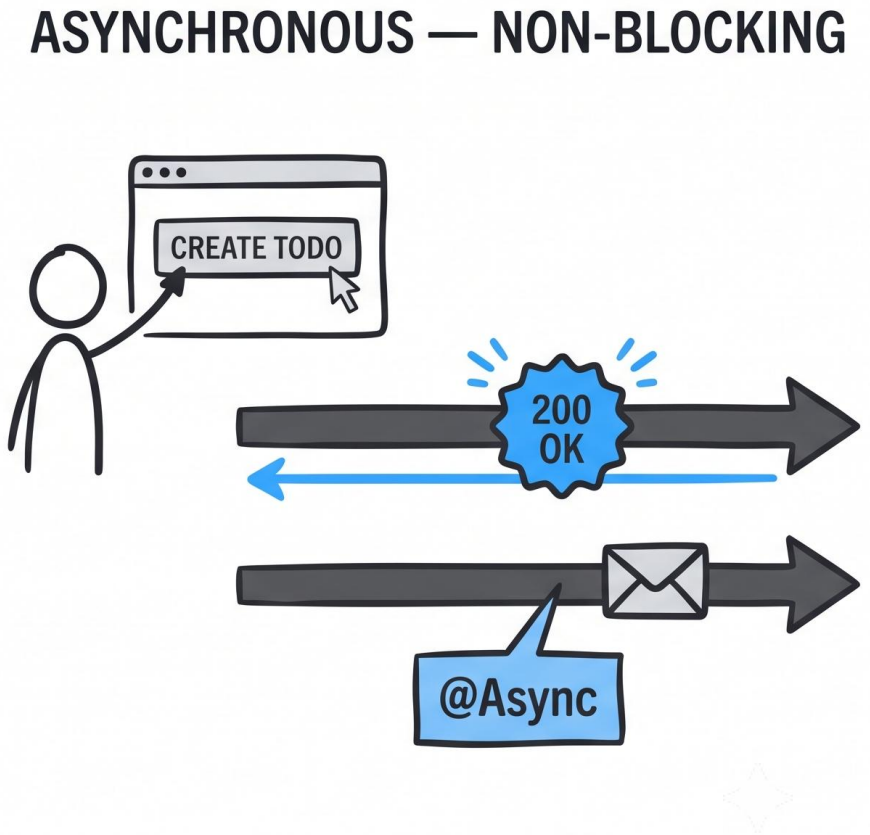
Preventing SMTP delays from blocking HTTP threads

The SMTP latency problem #1

- SMTP handshake and network transmission takes 1 to 3 seconds
- Synchronous sending freezes the user browser during form submit
- Ties up Tomcat request threads, reducing web server throughput
- Users experience laggy checkout, registration or submission

The SMTP latency problem #2





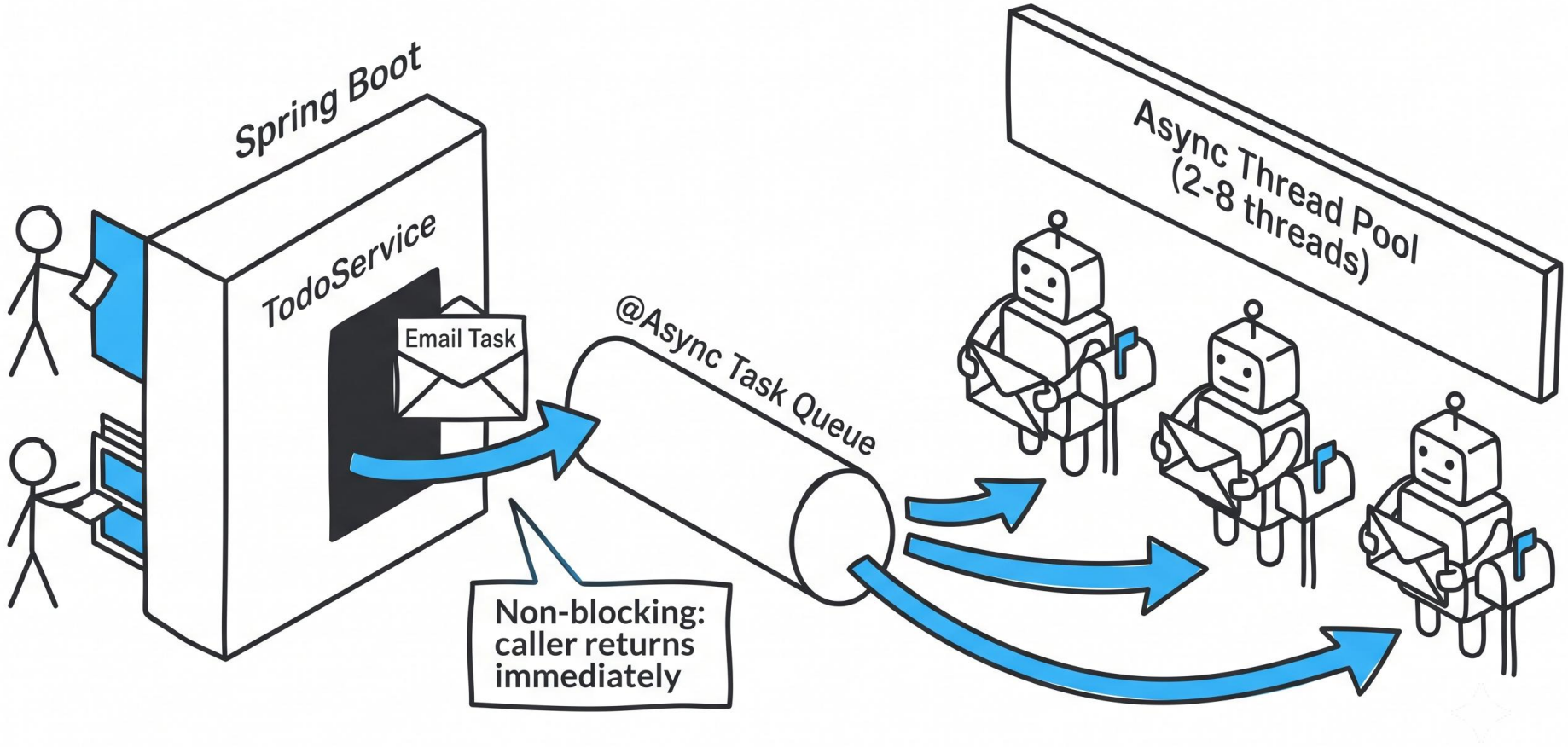
@EnableAsync and @Async

- @EnableAsync: enables asynchronous method execution in Spring
- @Async: marks email sending method to run in background thread
- Controller returns HTTP response immediately to user without waiting for email status.
- Mail task executes independently on a separate worker thread

Thread pool and task executors

- Configure ThreadPoolTaskExecutor with core and max pool sizes
- Set queue capacity to buffer incoming email dispatch tasks
- Prevents unconstrained thread creation under sudden traffic spikes
- Graceful shutdown ensures in-flight emails finish before restart

Async task queue



Error handling and production tips

- Async methods cannot propagate exceptions back to the HTTP caller
- Catch MailException inside service and log detailed errors
- Persist failed emails to a database table for scheduled retry
- Respect email provider rate limits to prevent account suspension

Q&A – Thank you